

LLM-Based Crypto Market Signal Detection from X

Technical University of Darmstadt
Multimedia Communications Lab, Summer Term 2026

Emircan Haslak
author1@mail.com

Yeseok Bae
author2@mail.com

Abstract—

NOTE: TODO's: Abstract, Referenzen, Grafiken, Methodik Teil, Evaluation, Conclusion

I. INTRODUCTION

The rapid growth of the cryptocurrency market has created a new form of financial speculation that is marked by unpredictability and strong sensitivity to external influences. Unlike traditional financial assets, cryptocurrencies are so heavily affected by public opinion and the actions of influential individuals that a single social media post can cause substantial changes in market prices. With cryptocurrencies now widely adopted, the ability to distinguish meaningful market signals from the large volume of online discussions has become essential for both risk management and market analysis. This project focuses on the challenge of identifying and classifying market relevant sentiment in social media content using Large Language Models (LLMs). By applying advanced natural language processing techniques, we aim to develop a scalable framework that links linguistic intent, categorized as "BUY," "SELL," or "NEUTRAL," to historical price movements. This approach provides a data driven perspective on how influential social media activity can affect cryptocurrency markets. The remainder of this paper is structured as follows. Section II defines the research problem and highlights the linguistic challenges of sentiment analysis in the cryptocurrency space. Section III outlines the system architecture and the core components of the detection pipeline. Section IV details the data acquisition process and the preprocessing steps required to format social media metrics. Section V explains the technical implementation, including the localized model deployment, iterative prompt engineering, and output normalization. Section VI interprets the model outputs by analyzing the generated reasoning and examining the disconnect between linguistic intent and actual market impact. Section VII presents the evaluation methodology, discusses performance metrics, and features a specific case study on influencer market dynamics. Finally, Section VIII summarizes the findings and suggests potential directions for future research.

II.

NOTE: PROBLEM STATEMENT

The growing number of cryptocurrency discussions on social media platforms has created a large and continuous flow of unstructured data. Although some of these posts contain

valuable market insights, most of the content consists of noise, such as spam, unfounded speculation, and irrelevant opinions. The main objective of this project is to automatically identify and extract meaningful market signals from this fast moving and highly dynamic data stream. One of the biggest challenges in this area is the unique language used within the cryptocurrency community. Traditional sentiment analysis methods, which often depend on standard dictionaries and predefined keywords, struggle to understand crypto-specific terms, memes, and slang expressions such as "HODL," and "to the moon,". In addition, discussions on platforms such as X frequently include sarcasm, irony, and humor. As a result, a post that appears positive at first glance may actually express criticism or negative sentiment. Determining whether a post suggests a "BUY," "SELL," or "NEUTRAL" position therefore requires a deeper understanding of context rather than relying on predefined sentiment dictionaries. Many existing approaches do not provide the level of semantic understanding needed to handle these linguistic complexities effectively. Moreover, there is a lack of frameworks that not only classify sentiment but also connect these classifications with historical price data to assess their actual influence on market behavior. This project seeks to address these limitations by using a Large Language Model capable of advanced contextual analysis. In doing so, it aims to establish a stronger connection between social media sentiment and measurable market trends.

III. SYSTEM DESIGN AND ARCHITECTURE

NOTE: TODO. THIS SECTION COULD HAVE MASSIVE CHANGES

The system was designed as a multi-stage processing pipeline that takes raw social media data, classifies it using a local LLM, standardizes the model's output, and evaluates the resulting signals against historical market data. The entire system is deployed on a local laboratory server, allowing full control over configuration, latency, and data processing without relying on external cloud services. The architecture consists of three main components: the data ingestion framework, the few-shot inference engine, and the price correlation evaluation module.

A. Data Extraction and Formatting

The first stage of the pipeline processes historical social media datasets that are organized by cryptocurrency ticker. Instead of analyzing text alone, the extraction module isolates

additional contextual information from each post to provide the language model with a richer input. For every entry, the system collects and processes the following information:

- **Textual Data:** The tweet content and the associated cryptocurrency ticker.
- **Temporal Information:** The exact timestamp of the post.
- **Engagement Metrics:** Public interaction data, including retweet count, like count, follower count, and reply count.

These attributes are extracted through a dedicated preprocessing script and mapped into structured variables. The variables are then inserted into a predefined prompt template, which is used during model inference.

B. Local Few-Shot Inference and Normalization Engine

The core processing layer is based on an OllamaLLM environment running a local instance of the Llama 3.1 model. The model is accessed through HTTP requests sent across the internal network to a dedicated server endpoint. To ensure consistent outputs, the model is configured to return responses in a strict JSON format. When a formatted few-shot prompt containing the tweet text and metadata is submitted, the model generates a JSON object containing the fields `signal`, `ticker`, `confidence`, and `reason`. Although the JSON constraint improves consistency, language models can still produce slight variations in their outputs. To address this issue, a normalization layer is applied immediately after inference. This component converts non-standard outputs such as "BUYSIGNAL", "BUY SIGNAL", or "B" into one of three predefined categories: **BUY**, **SELL**, or **NEUTRAL**. If the returned value does not match one of these categories or if the predicted ticker differs from the target asset, the system automatically assigns a fallback classification of **NEUTRAL**. The normalized results are then stored in a temporary CSV file for further evaluation.

C. Market Data Validation

The final stage of the system evaluates the accuracy of the generated signals by comparing them with historical cryptocurrency price data. The evaluation module combines the classification results stored in the temporary CSV file with a local database containing historical market prices. For each signal, the system calculates the price change over a predefined time window. Specifically, it calculates the difference between the asset's closing price on the day the tweet was posted and its closing price on the following trading day. The correctness of each signal is then determined using a set of predefined rules:

- **BUY Signal:** Considered correct if the asset's closing price increases on the following trading day.
- **SELL Signal:** Considered correct if the asset's closing price decreases on the following trading day.
- **NEUTRAL Signal:** Considered correct if the price change remains within 1% of the asset's closing price at the time the tweet was posted.

IV. DATA SOURCES AND PROCESSING

NOTE: Muss noch angepasst werden

The performance of the LLM in detecting market signals depends largely on the quality and relevance of the input data. To achieve reliable results, this project combines qualitative social media content with quantitative historical market data. Both data sources undergo preprocessing to ensure they are suitable for analysis and compatible with the inference engine.

A. Data Acquisition

To simulate realistic market conditions and capture authentic financial discussions, the system uses publicly available datasets instead of live API streams. The main source of textual data comes from several established datasets, including the Financial Tweets Dataset available on Hugging Face and the Cryptocurrency Sentiment Analysis Dataset provided by Mendeley Data. Together, these datasets offer a large collection of posts sourced from X (formerly Twitter), containing cryptocurrency references, market opinions, and investor sentiment. For focused case studies involving influential individuals, the project also uses a specialized dataset containing posts from Elon Musk's X account from February 2021, obtained through Kaggle. To evaluate the effectiveness of the LLM during the initial testing phase, a representative sample of tweets was selected for manual review and qualitative validation.

B. Data Preprocessing

The data preprocessing pipeline is designed to maintain the integrity of the original social media content, as contextual elements such as financial identifiers (cashtags like \$BTC), hashtags, and user mentions are crucial for accurate sentiment classification. Instead of aggressive noise reduction, which could strip away valuable context, the pipeline focuses on data structuring:

- **Data Mapping:** The raw JSON input files are parsed using a Python script to extract key metadata, including the user handle, publication date, retweet count, like count, reply count, and follower count.
- **Template Injection:** These attributes are mapped to a structured format and passed directly into the few-shot prompt template, ensuring the model receives a consistent input schema for every tweet.

C. Financial Time-Series Data

NOTE: TODO

To evaluate the real market impact of social media signals, the project incorporates historical cryptocurrency price data. This dataset contains detailed, time stamped records stored in a local CSV file, including the historical open and close prices for specific assets such as Bitcoin. The system relies on these financial records to verify the correctness of the generated signals. The timestamps of the price data are carefully aligned with those of the tweet datasets to create a consistent timeline. This allows the evaluation script to look up the exact opening

and closing prices on the day a tweet was posted, as well as the closing price of the next day, to measure the subsequent price movement.

V. IMPLEMENTATION

NOTE: Will probably have changes in the final weeks

The technical realization of the processing and evaluation pipeline is achieved through a modular Python codebase. The primary software stack consists of two separate execution scripts: `kom-ollama.py`, which manages model connectivity and signal extraction, and `evaluation.py`, which performs the data validation against financial tables.

A. Local Model Integration

NOTE: Vielleicht bei der Finalen Abgabe nicht dabei

The interface between the pipeline and the language model is implemented using the `langchain_ollama` library. Rather than maintaining a persistent socket connection to external cloud providers, communication with the local `llama3.1` model is driven via stateless HTTP POST requests. The client side initialization is handled by creating an `OllamaLLM` object configured with a specific loop structure:

```
llm = OllamaLLM(
    model="llama3.1",
    base_url="http://<server_ip>:11434",
    format="json"
)
```

B. Prompt Design and Refinement

The generation script utilizes the `pandas` framework to manage the structured processing of social media datasets. During the iteration loop, row attributes are extracted from the dataset and dynamically injected into a predefined prompt template for each entry.

The development of this prompt template followed an iterative refinement process to maximize the reliability of the language model. Initial testing utilized a basic zero-shot approach. However, this often resulted in formatting errors where the model generated invalid classifications such as `BUYSIGNAL` instead of the required `BUY` token. To mitigate this, the prompt was updated to a constrained zero-shot version that introduced explicit negative rules, specifically instructing the model to never output variations of the target categories.

The final implementation uses a structured few-shot inference template. This advanced prompt enforces strict JSON formatting rules and provides the model with three distinct contextual examples. These examples demonstrate how to handle institutional news, negative market events, and irrelevant discussions. Furthermore, the prompt forces the model to generate a confidence score and a logical reason for its classification, which are extracted and logged to ensure maximum transparency.

C. Output Parsing and Error Handling

NOTE: Wird noch angepasst

To defend the system against crashes caused by malformed text boundaries, the inference request is wrapped inside a conditional `try-except` block. Following a successful response, the script uses a localized normalization sequence to convert the unstructured string outputs into categorical data. The native string variable is converted to uppercase and stripped of leading or trailing whitespaces via the chain `str(signal).upper().strip()`. The code then executes a hard coded multi conditional match statement to repair inconsistent model generations:

```
if signal == "BUYSIGNAL" or
   signal == "BUY SIGNAL" or
   signal == "BU" or
   signal == "B":
    signal = "BUY"
if signal == "SELLSIGNAL" or
   signal == "SELL SIGNAL" or
   signal == "SEL" or
   signal == "SE" or
   signal == "S":
    signal = "SELL"
```

If the resulting token fails to match any criteria within the target tuple, a protective fallback rule automatically reassigns the token value to `NEUTRAL`.

D. Price Matching and Result Validation

NOTE: TODO

VI. EVALUATION

NOTE: TODO

VII. CONCLUSION AND FUTURE WORK

NOTE: TODO Conclusion

Future work should focus on improving the temporal resolution of the financial validation layer. Transitioning from daily closing prices to hourly or minute based price tracking would provide a much more accurate representation of the immediate market reaction following a social media post.

Furthermore, expanding the system to operate in a live environment represents a major avenue for future development. A theoretical connection to the official X (formerly Twitter) API is fully possible within the current system architecture and would enable real time signal generation. Unfortunately, the current enterprise pricing tiers for the required API access significantly exceeded the financial budget of this academic project. As a result, the live data integration remains a theoretical concept that could be readily implemented in future iterations provided that appropriate funding is secured.